

SISTEMA MedIA

Expediente Clínico Electrónico — Distrito de Salud I, Chiapas

DOCUMENTO 4

Infraestructura y Despliegue

Docker · Azure App Service · PostgreSQL · Static Web Apps · CI/CD

Versión	1.0 — Inicial
Plataforma	Microsoft Azure (East US)
Stack	FastAPI · PostgreSQL · React + Vite
Ciclo escolar	8° Semestre IDTS — UNACH, Enero–Junio 2026

1. Entornos del Sistema

MedIA opera en dos entornos bien diferenciados: desarrollo local con Docker Compose y producción en Microsoft Azure. Esta separación garantiza paridad de configuración y facilita la detección temprana de errores antes de cualquier despliegue a producción.

1.1 Variables de Entorno — Desarrollo vs. Producción

Las credenciales y configuraciones sensibles nunca se hardcodean en el código fuente. Todas se gestionan mediante archivos `.env` (desarrollo) o Azure App Service Settings (producción), conforme al Requisito Forense 8 del proyecto.

Variable	Desarrollo (.env)	Producción (Azure)
DATABASE_URL	postgresql://media_user:dev_pass@postgres:5432/media_dev	postgresql://<admin>:<pass>@media-pg.postgres.database.azure.com:5432/media_prod?sslmode=require
SECRET_KEY	dev-secret-key-insegura-local	Generada con <code>openssl rand -hex 32</code> , almacenada en Key Vault
ENVIRONMENT	development	production
AZURE_STORAGE_CONN_STR	(no aplica)	DefaultEndpointsProtocol=https;AccountName=...
TOTP_ISSUER	MedIA-Dev	MedIA
JWT_EXPIRATION_MIN	60	30
CORS_ORIGINS	http://localhost:5173	https://media-chiapas.azurestaticapps.net
BLOB_SAS_TTL_MIN	N/A	15
LOG_LEVEL	DEBUG	WARNING

IMPORTANTE: El archivo `.env` nunca se incluye en el repositorio Git. Se agrega a `.gitignore` desde la inicialización del proyecto. En producción, todas las variables se configuran directamente en Azure App Service > Configuration > Application Settings.

2. Configuración Docker para Desarrollo

El entorno de desarrollo se levanta con un único comando (`docker compose up -d`) y provee PostgreSQL, la API FastAPI y Adminer para administración de base de datos. El frontend React se ejecuta en modo hot-reload directamente con Vite fuera de Docker para mayor velocidad de desarrollo.

2.1 docker-compose.yml (desarrollo)

```
# docker-compose.yml - Entorno de desarrollo
# Media ECE Chiapas v1.0

version: '3.9'

services:
  postgres:
    image: postgres:15-alpine
    container_name: media_postgres
    restart: unless-stopped
    environment:
      POSTGRES_USER: media_user
      POSTGRES_PASSWORD: dev_pass_changeme
      POSTGRES_DB: media_dev
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./database:/docker-entrypoint-initdb.d:ro
    ports:
      - '5432:5432'
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U media_user -d media_dev']
      interval: 10s
      timeout: 5s
      retries: 5

  # DESARROLLO: backend corre fuera de Docker. Comando: uvicorn app.main:app --reload
  # Frontend corre fuera de Docker. Comando: npm run dev

  adminer:
    image: adminer:latest
    container_name: media_adminer
    restart: unless-stopped
    ports:
      - '8080:8080'
    depends_on:
      - postgres

volumes:
  postgres_data:
    driver: local
```

El volumen `postgres_data` persiste los datos entre reinicios del contenedor. El directorio `./database` se monta como read-only para que Docker ejecute automáticamente los scripts SQL en orden lexicográfico al primer arranque (`01_schema.sql`, `02_indices.sql`, `03_seeds.sql`).

2.2 Dockerfile del Backend (FastAPI + Python 3.11)

```
# backend/Dockerfile
FROM python:3.11-slim

WORKDIR /app

# Instalar dependencias del sistema
RUN apt-get update && apt-get install -y \
    libpq-dev gcc && \
    rm -rf /var/lib/apt/lists/*

# Copiar requirements primero (cache de capas)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copiar código fuente
COPY . .

# Crear usuario no-root para seguridad
RUN adduser --disabled-password --gecos '' appuser && \
    chown -R appuser:appuser /app
USER appuser

EXPOSE 8000

# Uvicorn con workers según CPU disponibles
CMD ["uvicorn", "main:app", "--host", "0.0.0.0",
    "--port", "8000", "--workers", "2",
    "--proxy-headers", "--forwarded-allow-ips="]
```

2.3 Dockerfile del Frontend (React + Vite + Node 20)

```
# frontend/Dockerfile
# Etapa 1: Build
FROM node:20-alpine AS builder

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci --frozen-lockfile

COPY . .
RUN npm run build

# Etapa 2: Servir con Nginx
FROM nginx:alpine

COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

2.4 docker-compose.prod.yml (Stack Completo para Azure)

Este archivo se utiliza para pruebas de integración del stack completo previas al despliegue en Azure. No se usa directamente en producción ya que cada componente corre en su servicio Azure dedicado.

```
# docker-compose.prod.yml - Stack completo
version: '3.9'

services:
  backend:
    image: mediachiapas.azurecr.io/media-backend:latest
    restart: always
    env_file: .env.prod
    ports:
      - '8000:8000'
    environment:
      - ENVIRONMENT=production
    deploy:
      replicas: 2
      resources:
        limits:
          cpus: '1.0'
          memory: 512M

  frontend:
    image: mediachiapas.azurecr.io/media-frontend:latest
    restart: always
    ports:
      - '80:80'
    depends_on:
      - backend

# PostgreSQL en produccion = Azure Database for PostgreSQL
# No se containeriza. Se usa la cadena de conexion de Azure.
```

3. Pasos de Despliegue en Azure

La arquitectura de producción utiliza servicios gestionados de Azure para maximizar la disponibilidad y minimizar la carga operativa del equipo. El despliegue sigue el siguiente orden de aprovisionamiento.

3.1 Servicios Azure Utilizados

Servicio Azure	Uso en Media	Tier Recomendado
Azure App Service	Backend FastAPI (API REST)	B1 (1 vCore, 1.75GB RAM) — Básico (prueba gratuita y demo)
Static Web Apps	Frontend React/Vite compilado	Free (incluido en cuenta gratuita)
Database for PostgreSQL	Base de datos principal (40 tablas)	Burstable B1ms (1 vCore, 2GB) — dev/test
Blob Storage	PDFs externos de laboratorio y documentos de tutores (únicos archivos de origen externo)	LRS Standard — Tier Hot
Front Door + CDN	Enrutamiento global, cache de catálogos	Standard (o Azure CDN Classic)
Container Registry	Repositorio de imágenes Docker	Basic (incluye builds gratuitos)
Key Vault	Secretos y cadenas de conexión	Standard
Azure Monitor	Alertas, logs, métricas de incidentes	Incluido en servicios anteriores

3.2 Paso 1 — Crear Grupo de Recursos

```
# Azure CLI — Crear grupo de recursos
az group create \
  --name rg-media-chiapas \
  --location eastus

# Nota: East US tiene menor latencia desde CDMX/Chiapas
# que West US o South Central US.
```

3.3 Paso 2 — Azure Database for PostgreSQL

```
# Crear servidor PostgreSQL flexible
az postgres flexible-server create \
  --resource-group rg-media-chiapas \
  --name media-pg-server \
  --location eastus \
  --admin-user mediaadmin \
```

```
--admin-password '<PASSWORD_SEGURO>' \  
--sku-name Standard_Blms \  
--tier Burstable \  
--version 16 \  
--storage-size 32 \  
--backup-retention 14 \  
--geo-redundant-backup Disabled  
  
# Crear base de datos  
az postgres flexible-server db create \  
  --resource-group rg-media-chiapas \  
  --server-name media-pg-server \  
  --database-name media_prod  
  
# Habilitar firewall para App Service  
az postgres flexible-server firewall-rule create \  
  --resource-group rg-media-chiapas \  
  --name media-pg-server \  
  --rule-name allow-app-service \  
  --start-ip-address 0.0.0.0 \  
  --end-ip-address 0.0.0.0
```

3.4 Paso 3 — Azure Blob Storage

```
# Crear cuenta de almacenamiento  
az storage account create \  
  --name mediablobstorage \  
  --resource-group rg-media-chiapas \  
  --location eastus \  
  --sku Standard LRS \  
  --kind StorageV2 \  
  --access-tier Hot  
  
# Crear contenedor para archivos externos (laboratorio y tutores)  
az storage container create \  
  --name documentos-externos \  
  --account-name mediablobstorage \  
  --public-access off  
  
# Los SAS tokens se generan desde el backend (15 min TTL)  
# Nunca se expone la cadena de conexion al frontend.
```

3.5 Paso 4 — Azure App Service (Backend FastAPI)

```
# Crear plan App Service  
az appservice plan create \  
  --resource-group rg-media-chiapas \  
  --name media-backend-plan \  
  --location eastus \  
  --sku B1 \  
  --is-linux  
  
# Crear Web App con imagen Docker  
az webapp create \  
  --resource-group rg-media-chiapas \  
  --plan media-backend-plan \  
  --name media-backend-api \  
  --deployment-container-image-name mediachiapas.azurecr.io/media-backend:latest
```

```
# Configurar variables de entorno
az webapp config appsettings set \
  --resource-group rg-media-chiapas \
  --name media-backend-api \
  --settings \
    DATABASE_URL='postgresql://...' \
    SECRET_KEY='@Microsoft.KeyVault(...)' \
    ENVIRONMENT='production' \
    BLOB_CONNECTION_STR='@Microsoft.KeyVault(...)'
```

3.6 Paso 5 — Azure Static Web Apps (Frontend React)

```
# Crear Static Web App (deploy desde GitHub Actions)
az staticwebapp create \
  --resource-group rg-media-chiapas \
  --name media-frontend \
  --location eastus2 \
  --source https://github.com/equipo/media-chiapas \
  --branch main \
  --app-location /frontend \
  --output-location dist \
  --login-with-github

# Azure genera automaticamente un workflow de GitHub Actions
# que hace build y deploy en cada push a main.
```

3.7 Paso 6 — Azure Front Door + CDN

Azure Front Door actúa como capa de enrutamiento global. Distribuye el tráfico del frontend hacia Static Web Apps y los catálogos estáticos (CIE-10, municipios INEGI) se cachean en los nodos CDN más cercanos a Chiapas, reduciendo la latencia en clínicas rurales con conectividad limitada.

Estrategia de Cache para Catálogos (Cómputo Distribuido)

cat_cie10, cat_medicamentos, cat_lenguas_indigenas → Cache CDN con TTL 24h

cat_estados, cat_municipios, cat_localidades → Cache CDN con TTL 7 días

Invalidación manual desde endpoint /admin/cache/invalidate (rol SUPERADMIN)

Notas médicas y datos de pacientes → NUNCA se cachean (datos clínicos sensibles)

Catálogos cacheados en memoria durante la sesión activa. IndexedDB descartado del MVP — queda como mejora futura

4. Plan de Prueba Azure con Cuenta Gratuita — Semanas 4-5

La cuenta gratuita de Azure proporciona créditos de USD \$200 durante 30 días y servicios gratuitos por 12 meses. El siguiente plan minimiza el consumo de créditos para las semanas de prueba del proyecto.

Semana	Actividad	Servicio Azure	Costo estimado
Semana 4, Día 1	Crear grupo de recursos, PostgreSQL, Storage	Database B1ms + Storage LRS	~\$0.02/hora
Semana 4, Día 2-3	Ejecutar migraciones SQL, cargar seeds	PostgreSQL activo	~\$0.30/día
Semana 4, Día 4-5	Deploy backend FastAPI en App Service	App Service B1 (free tier 30 días)	USD \$0 (free tier)
Semana 5, Día 1-2	Deploy frontend en Static Web Apps	Static Web Apps Free	USD \$0 (free siempre)
Semana 5, Día 3-4	Pruebas end-to-end login 2FA + consulta	App Service + PostgreSQL + Blob	~\$0.50/día
Semana 5, Día 5	Activar Azure Monitor, validar alertas	Monitor (incluido)	USD \$0

Recomendación de Costo para Cuenta Gratuita

Apagar PostgreSQL flexible server fuera de horario de pruebas (az postgres flexible-server stop)

Usar SKU B1ms para PostgreSQL (más barato que B2ms, suficiente para pruebas)

Configurar alerta de presupuesto en Azure Cost Management: umbral USD \$50

Static Web Apps Free es permanente — no consume créditos de prueba

GitHub Actions para CI/CD es gratuito con repositorio público o 2000 min/mes privado

5. Procedimiento PITR (Point-in-Time Recovery) en Azure PostgreSQL

Azure Database for PostgreSQL Flexible Server realiza respaldos completos automáticamente (una vez por semana) y WAL (Write-Ahead Log) cada 5 minutos, permitiendo restaurar a cualquier punto en el tiempo dentro del periodo de retención configurado (14 días en MedIA).

5.1 Procedimiento de Recuperación mediante Azure Portal

1. Acceder a Azure Portal: portal.azure.com → Grupo de recursos rg-media-chiapas → media-pg-server
2. En el menú lateral, seleccionar Overview → Restore
3. Configurar los parámetros de restauración:
 - Restore point type: Specific point in time
 - Restore to: Seleccionar fecha y hora exacta del punto de recuperación (formato UTC)
 - Server name: media-pg-server-restored (NUNCA sobrescribir el servidor de producción)
4. Hacer clic en Review + Create → Create
5. Esperar entre 15-45 minutos dependiendo del tamaño de la base de datos
6. Una vez restaurado, ejecutar validaciones:

```
# Conectar al servidor restaurado y validar integridad
psql
'postgresql://mediaadmin:<pass>@media-pg-server-restored.postgres.database.azure.com/media_prod'

-- Verificar numero de registros críticos
SELECT COUNT(*) FROM auditoria_accesos;
SELECT COUNT(*) FROM notas_medicas;
SELECT MAX(creado_en) FROM encuentros_clinicos;
```

7. Si la validación es exitosa, actualizar DATABASE_URL en App Service para apuntar al servidor restaurado
8. Documentar el incidente en bitacora_recuperacion con: fecha_inicio, fecha_fin, punto_restauracion, responsable, registros_validados

5.2 Procedimiento mediante Azure CLI

```
# Restaurar a punto en el tiempo (PITR)
az postgres flexible-server restore \
  --resource-group rg-media-chiapas \
  --name media-pg-server-restored \
  --source-server media-pg-server \
  --restore-time '2026-03-15T14:30:00Z' \
  --location eastus

# Verificar estado de la restauracion
az postgres flexible-server show \
```

```
--resource-group rg-media-chiapas \  
--name media-pg-server-restored \  
--query 'state'
```

Tabla de Seguimiento — bitacora_recuperacion

Campos: id_recuperacion (UUID PK), tipo_evento (PITR/FAILOVER/BACKUP_RESTORE),
fecha_inicio_incidente (TIMESTAMPTZ), fecha_fin_recuperacion (TIMESTAMPTZ),
punto_restauracion (TIMESTAMPTZ), registros_validados (INTEGER),
responsable_id (FK usuarios_sistema), notas_tecnicas (TEXT)

Esta tabla cumple Requisito Forense 1: registro inmutable de eventos de recuperación.

6. Configuración de Azure Monitor para Alertas de Incidentes

Azure Monitor integra métricas de App Service, PostgreSQL y Blob Storage en un panel centralizado. Las alertas se configuran para notificar al equipo de seguridad (rol AUDITOR_SEGURIDAD) y al superadmin ante eventos que requieren atención inmediata.

6.1 Alertas Configuradas

Alerta	Condición	Severidad	Acción
CPU App Service > 80%	CPU > 80% por 5 min	Sev 2 — Warning	Email + Slack webhook
Memoria App Service > 85%	Memory > 85% por 3 min	Sev 2 — Warning	Email al equipo
PostgreSQL conexiones > 90%	Active connections > 90% del máximo	Sev 1 — Critical	Email + SMS responsable
HTTP 5xx > 10 en 5 min	Failed requests > 10	Sev 1 — Critical	PagerDuty / Teams alert
Storage latencia > 500ms	Blob avg latency > 500ms	Sev 3 — Info	Log en Monitor
Intentos login fallidos > 20	Métrica custom desde auditoria_accesos	Sev 1 — Critical	Bloqueo automático + alerta

6.2 Configurar Alerta CPU via Azure CLI

```
# Crear action group para notificaciones
az monitor action-group create \
  --resource-group rg-media-chiapas \
  --name media-alerts-team \
  --short-name media-alerts \
  --email email=admin@unach.edu name=Administrador

# Crear alerta de CPU
az monitor metrics alert create \
  --resource-group rg-media-chiapas \
  --name alert-cpu-backend \
  --scopes /subscriptions/<SUB_ID>/resourceGroups/rg-media-chiapas/providers/Microsoft.Web/sites/media-backend-api \
  --condition 'avg Percentage CPU > 80' \
  --window-size 5m \
  --evaluation-frequency 1m \
  --action media-alerts-team \
  --severity 2
```

6.3 Log Analytics Workspace — Consultas Forenses

Se configura un Log Analytics Workspace para centralizar los logs de App Service y correlacionarlos con los registros de auditoria_accesos de PostgreSQL. Esto cumple el Requisito Forense 1 (Registro de eventos) y 3 (Trazabilidad).

```
// Kusto Query Language – Detección de accesos anómalos
AppRequests
| where TimeGenerated > ago(1h)
| where ResultCode == '403'
| summarize Count = count() by ClientIP, bin(TimeGenerated, 5m)
| where Count > 10
| order by Count desc
```

Esta consulta detecta IPs que generan más de 10 respuestas 403 en ventanas de 5 minutos, señal de posible ataque de fuerza bruta o escaneo de permisos. El resultado alimenta el módulo de incidentes de seguridad de MedIA.

7. Resumen de Arquitectura de Despliegue

La siguiente tabla consolida todos los componentes del sistema MedIA en producción, confirmando la consistencia entre la infraestructura Azure y el modelo de base de datos de 40 entidades descrito en el Reporte Técnico v3.

Componente	Tecnología	Servicio Azure	Estado
Frontend	React 18 + Vite + shadcn/ui	Static Web Apps Free	Producción
Backend API	FastAPI + Python 3.11	App Service B1 Linux	Producción
Base de datos	PostgreSQL 16 (40 tablas)	Database for PostgreSQL B1ms	Producción
Documentos PDF	Blob Storage LRS	Storage Account Standard	Producción
CDN / Enrutamiento	Azure Front Door	Front Door Standard	Producción
Secretos	Azure Key Vault	Key Vault Standard	Producción
Monitoreo	Azure Monitor + Log Analytics	Incluido en servicios	Producción
CI/CD	GitHub Actions	Integrado con Static Web Apps	Producción